

할로우 나이트 모작

엔진: Unity 2021.03.25f1 2D Core

장르: 2D, 플랫폼머, 메트로베니아, 액션

개발기간: 2023.05. ~ 2023.06. (1 개월)

GitHub 주소: <https://github.com/Nerbe72/HollowKnight-Imitation>



게임 설명

주인공인 사슴벌레가 진실을 향해 나아가는 메트로베니아 액션 게임입니다. 해당 게임에서 이용하는 플랫폼머 장르의 입력방식과 벽 짚기, 보스의 패턴 구현, 부적을 이용한 장비 시스템, 물건을 구매하는 재화 및 상점 시스템 등을 모작/구현합니다.

개발 목표

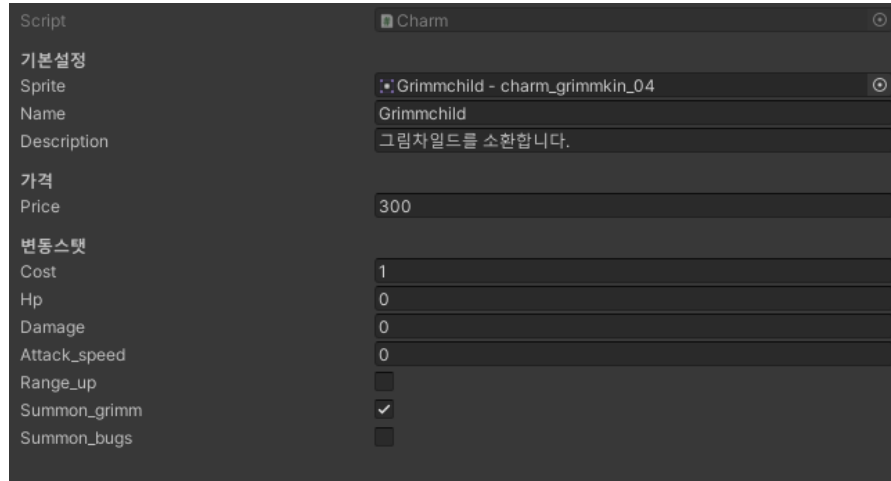
1. 부적(장비) / 실시간 스탯 변화
2. 재화 시스템 및 재화
3. InputSystem 을 활용한 이동 / 공격 / 회피
4. 착용 가능한 부적(장비)을 구매
5. 보스의 다양한 패턴과 영구적인 사망 처리
6. 플레이어 공격과 장애물 사이의 밀쳐내는 상호작용
7. 미니맵 기능
8. 벽 미끄러짐 및 고정된 방향성을 가지는 벽 점프
9. 자체 중력 시스템을 이용한 이동 및 점프
10. 디버그 모드를 통한 코드 통제 / 시스템 최적화

기술 요약

부적 시스템

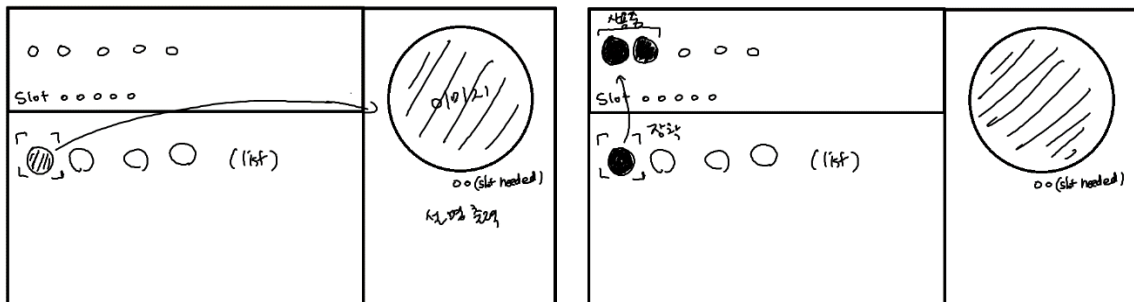
부적을 착용하고 해제하는 시스템을 구현하였다.

부적은 scriptableObject 를 이용해 부적별로 다양한 기능을 수행할 수 있도록 하였다.

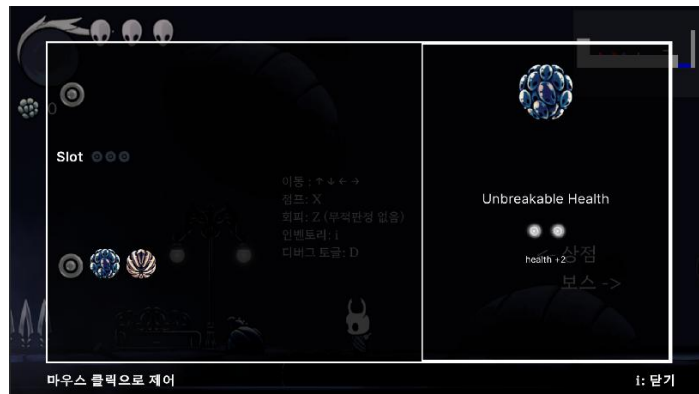


일반적인 상황에서는 창을 열어도 수정이 불가능하며, 맵에 존재하는 벤치에 앉은 경우에만 수정이 가능하도록 구현하였다. 기본적인 체력 공격력 등의 변화부터 소환수를 소환하는 등의 기능을 간편하게 에셋으로 생성 및 수정할 수 있다.

장착한 부적의 비용만큼 코스트가 증가하며, 마지막 착용한 부적에 한해 코스트가 부족해도 착용할 수 있는 대신 받는 데미지가 2 배가 되는 오버로드 상태도 구현되었습니다.



매 장황. 해제하면 Apply Charm(); 호출.



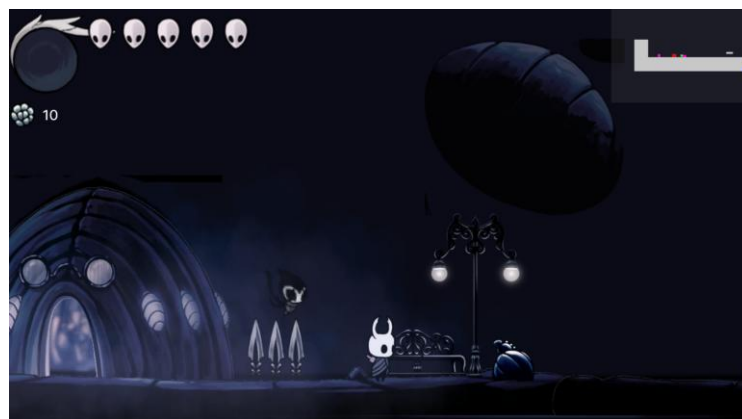
위는 구현을 위한 대략적인 구현 방향이며 하단은 실제 인게임 구현 화면입니다.



의자에 앉으면 자동으로 모든 체력을 회복하며, 부적을 수정할 수 있는 상태가 됩니다.



빨간색 부적은 소환수를 소환하는 부적이며, 파란색 부적은 최대 체력을 올려주는 부적입니다. 장착 즉시 반영되어 추종자가 생기고 체력이 늘어난 것을 확인할 수 있습니다.



모든 장착 반영은 장착 애니메이션이 끝나는 즉시 한번, 해제 애니메이션이 끝나는 즉시 한번 호출되도록 하여 불필요한 연산을 최소화 했습니다.

개발 중 발생한 문제

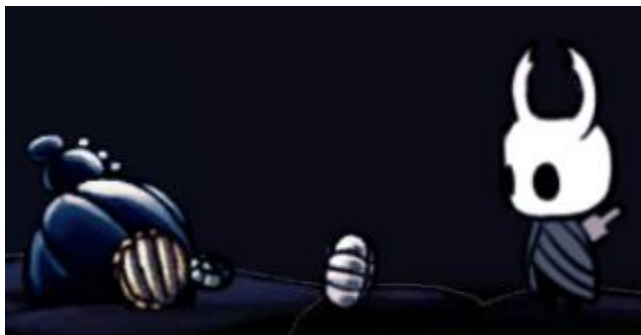
1.처음 생성시 부적 위치가 지정한 위치에 정확히 배치되지 않는 문제

해결

1.라이프 사이클상 GUI가 그려지지 않은 상태에서 위치를 지정하기 때문에 생긴 문제이므로 위치 지정을 1프레임 뒤로 미뤄 해결

```
public enum GeoAmount
{
    One = 1,
    Silver = 10,
    Gold = 20
}
```

재화는 1, 10, 20을 상승시키는 세가지 재화를 가지고 있으며 이는 GeoAmount라는 enum으로 관리됩니다. 이미지 역시 이 enum에 맞춰 변경될 수 있도록 설정하였습니다.



재화를 공급할 수 있는 돌을 제작하여 게임 내에서 직접 재화를 벌어들일 수 있습니다.

무조건 하나 이상의 재화를 드랍하도록 설정하였으며, 디폴트는 1Geo 입니다.

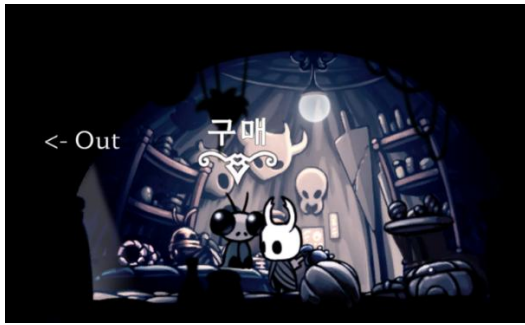
Random함수를 통해 Silver혹은 Gold가 드랍될 수 있으며 종류에 상관없이 최대 5개가 드랍될 수 있도록 하였습니다.

단순히 생성되는 것이 아닌 공중으로 잠시 솟았다 떨어지도록 했습니다.

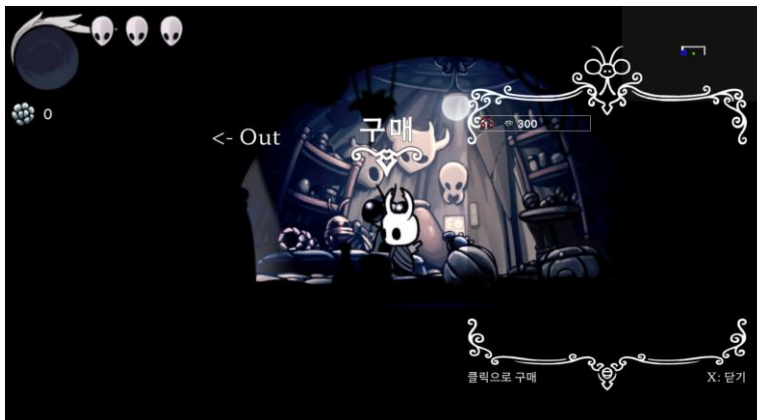
이 때, 재화는 단순히 바닥에 떨어지지 않고 최대 2회 바닥에서 튕기도록 했습니다.

```
if (collision.collider.CompareTag(GameTagMask.Tag(Tags.Ground)))
{
    if (bounceCount > 0)
    {
        maxVelocity *= -1 * bounce; // * -0.6f
        m_rigidbody.velocity = maxVelocity;
        bounceCount--;
    }
}
```

자연스럽게 두 번 튕길 수 있도록 바닥에 떨어질 시 velocity에 -0.6을 곱하여 다시 위로 약간 솟아오르도록 하였고, bounceCount변수와 bounce 변수를 따로 두어 쉽게 수정 가능하도록 했습니다.



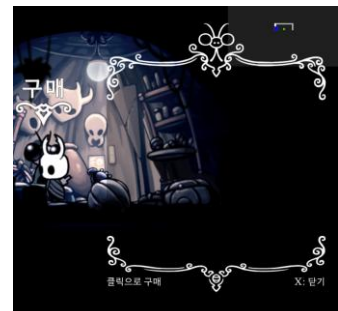
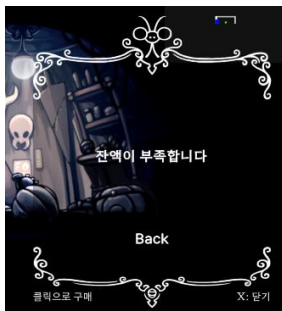
이렇게 벌어들인 재화는 상점에서 이용 가능합니다.



상점에서 원하는 부적을 선택하고 구매할 수 있습니다.

재화가 충분하다면 구매에 성공하고 부적 목록에서 소지하고 있음을 확인할 수 있습니다.

하지만 재화가 부족하다면 재화가 부족하다는 문구를 띄우며 구매되지 않습니다.



구매 완료하면 상점 목록에서도 제거되며, 소지한 부적임을 나타내는 변수값이 true가 됩니다.

부적 소지 목록은 상점을 다시 이용할 수 있도록 게임을 재시작하면 초기화 되도록 했습니다.

개발 중 발생한 문제

1. toolkit 이용시 sprite 애니메이션을 추가하기 위한 방안

해결방안

```
private void SwapSprites(FramePos _pos)
{
    switch (_pos)
    {
        case FramePos.Top:
            if (m_frameCheck.top >= m_frameCount.top)
            {
                m_frameCheck.top = 0;
                m_frameTask.top.Pause();
                return;
            }
            m_topFrame.style.backgroundImage = new StyleBackground(m_topSprites[m_frameCheck.top]);
            m_frameCheck.top++;
            break;
        case FramePos.Middle:
            if (m_frameCheck.middle >= m_frameCount.middle)
            {
                m_frameCheck.middle = 0;
                m_frameTask.middle.Pause();
                return;
            }
            m_middleFrame.style.backgroundImage = new StyleBackground(m_middleSprites[m_frameCheck.middle]);
            m_frameCheck.middle++;
            break;
        case FramePos.Bottom:
            if (m_frameCheck.bottom >= m_frameCount.bottom)
            {
                m_frameCheck.bottom = 0;
                m_frameTask.bottom.Pause();
                return;
            }
            m_bottomFrame.style.backgroundImage = new StyleBackground(m_bottomSprites[m_frameCheck.bottom]);
            m_frameCheck.bottom++;
            break;
    }
}
```

1.IVisualElementScheduledItem을 이용하여 top mid bottom 애니메이션을 분리한 뒤 각각 정해진 시간마다 이미지를 변경하는 방식으로 애니메이션을 추가했습니다. 이 애니메이션은 sprite 배열에 저장된 이미지를 하나씩 불러오며 배열의 마지막에 도착하면 애니메이션을 중지하고 현 상태를 유지합니다.

기술 요약

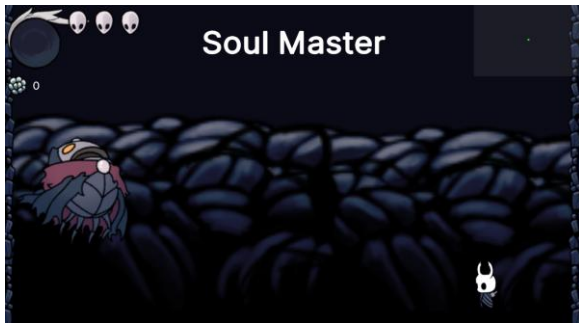
보스 패턴 / 사망처리

패턴은 본게임을 참고하여 3가지 및 대기 패턴을 만들었습니다.

총 4가지의 패턴은 Random.Range함수에 의해 비슷한 확률로 패턴을 진행하게 됩니다.

```
private enum Pattern
{
    A,
    B,
    C,
    D,
    Count
}
참조 4개
private enum Warp
{
    Start,
    End,
}
```

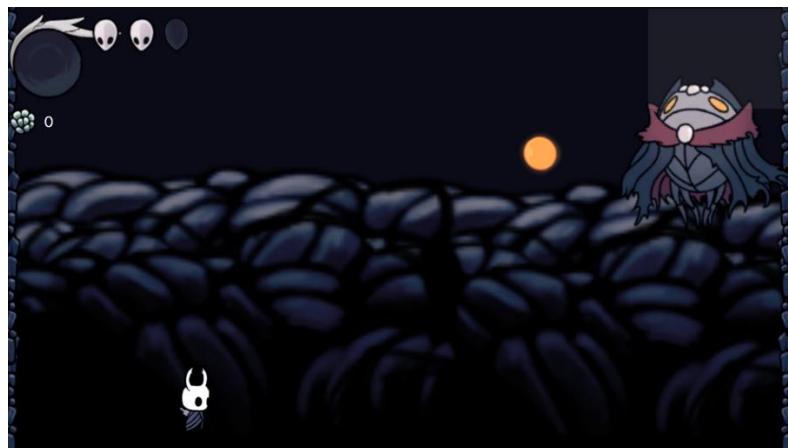
해당 보스는 4가지 패턴과 순간이동을 수행하기에 enum으로 구성하여 정해진 동작만 수행할 수 있도록 작성했습니다.



보스맵에 진입하면 추락하게 되고 플레이어가 지면에 도달하면 이름을 출력하고 화면이 흔들리며 플레이어의 이동을 잠시 막습니다.

패턴을 지정하고 그 패턴에 해당하는 위치로 보스가 이동하게 되며 이를 포함하여 각 패턴의 정보를 나열하면 이렇습니다.

패턴A – 유도 투사체



- 1.플레이어와 좌, 우측 지정 위치 사이의 거리를 비교하여 더 먼 장소로 순간이동.
- 2.플레이어에게 유도체를 3회 발사합니다. 유도체는 플레이어와 유도체의 외적값을 각속도로

이용하여 플레이어를 향합니다.

패턴B - 휩쓸기

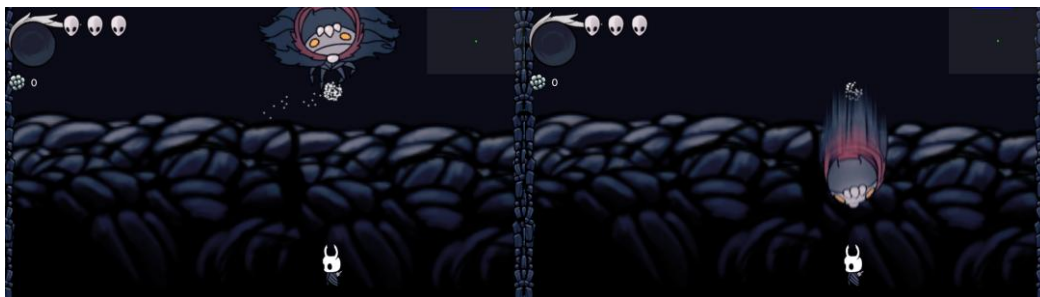


1.플레이어와 좌 우측 지정 위치 사이의 거리를 비교하여 더 먼 위치로 순간이동

2.반대 지정 위치까지 지면을 따라 이동

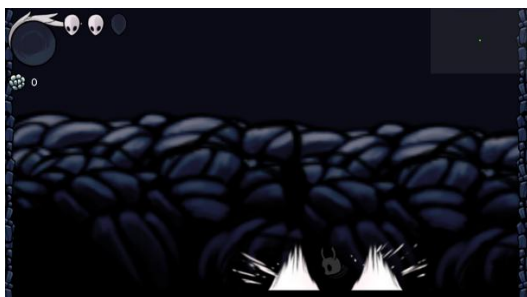
적에게 피격 판정이 있기에 점프 대쉬 혹은 점프 하단공격으로 회피하는 패턴입니다.

패턴C - 지면 강타



1.플레이어 위치 위쪽으로 순간이동

2.위로 살짝 떴다가 지면으로 추락



3.지면에 도달하면 좌우로 이동하는 충격파를 생성합니다.

충격파 또한 개별 피격 판정이 있습니다.

해당 웨이브는 OnBecomeInvisible함수를 통해 보이지 않는 화면 밖으로 벗어나면 제거됩니다.

패턴D - 휴식

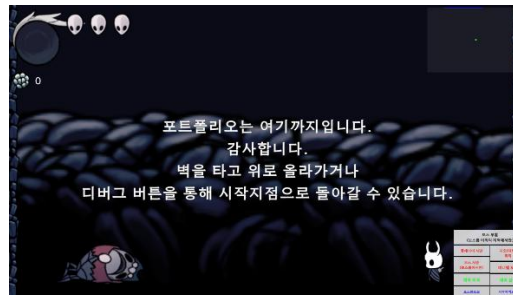
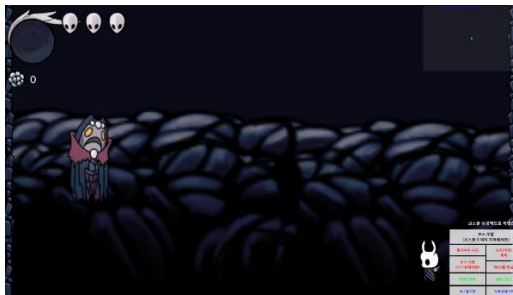


맵에 지정된 중앙 위치에서 대기합니다.

플레이어가 잠시 쉬어가거나 확실한 딜 타이밍으로 이용할 수 있습니다.

지정된 대기 시간 (1초)만큼 대기하지만 중간에 플레이어에게 공격받으면 즉시 해당 패턴을 종료합니다.

보스 사망처리



보스는 사망하면 그 자리에서 쪼그라들며 바닥에 추락하고

지면에 도달하면 생기를 잃은 채 누워있게 됩니다.

보스 사망은 **게임을 껐다 켜도 유지**되며, 디버그 모드를 통해 다시 부활시킬 수 있습니다.

보스맵 진입시 이미 사망했는지 확인하며 이미 사망했다면 곧바로 추락 및 애니메이션 재생 속도를 빠르게 한 뒤 사망 동작을 진행합니다.

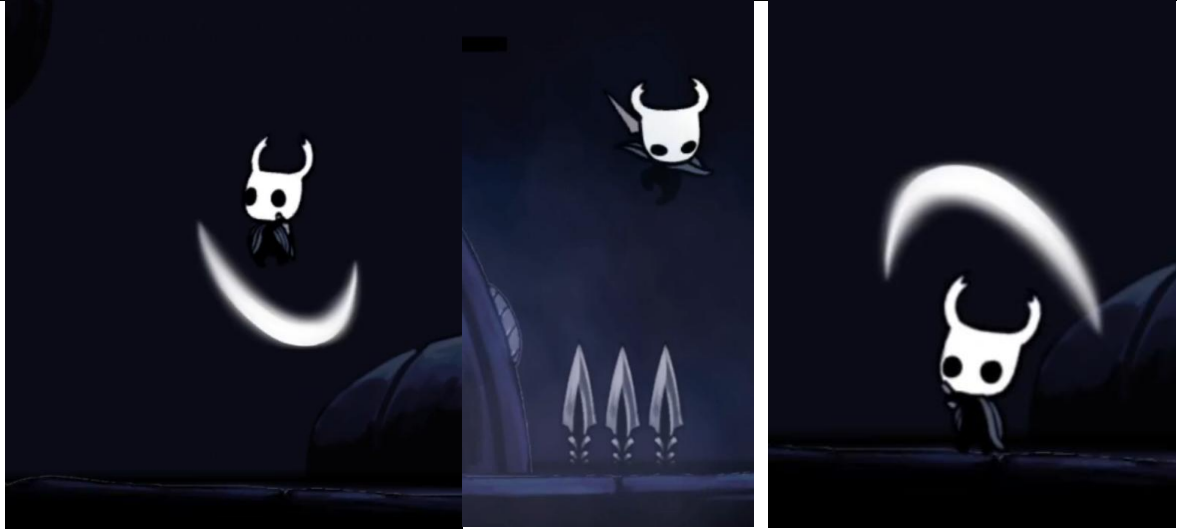
이 동작을 수행하는 동안 플레이어는 맵 내에서 추락중이기 때문에 해당 장면을 확인할 수 없다는 점을 이용하여 코드 수정을 최소화하여 보스가 이미 사망한 상태를 구현해 보았습니다.

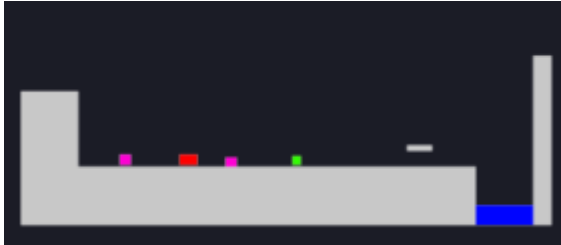
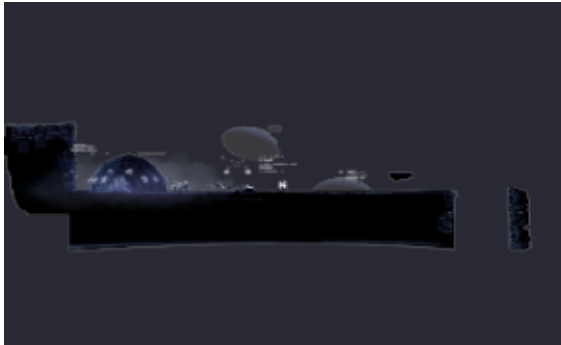
개발 중 발생한 문제

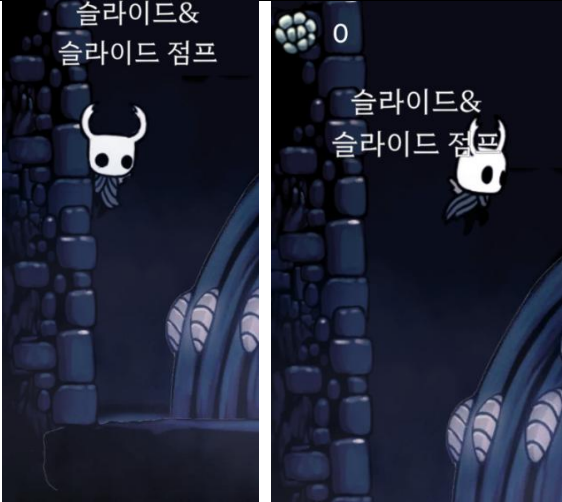
1.보스가 순간 이동할 때 바로 transform.position을 변경하면 보스가 이동할 때 만들어지는 파티클이 한 점에서만 머무르는 문제

해결방안

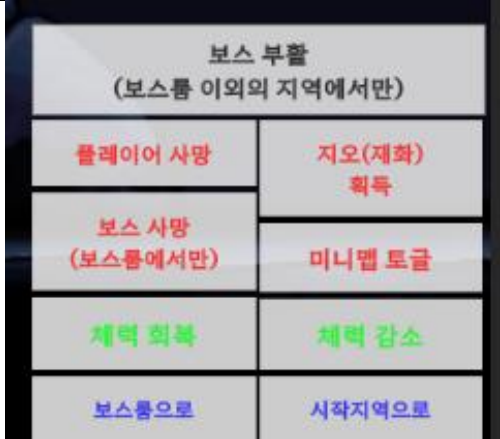
1.보스의 순간이동을 직접적인 위치 변경 대신 Lerp를 이용하여 sprite와 collider를 끈 채 빠르게 이동하는 방식으로 변경

기술 요약	플레이어 공격 상호작용
 플레이어는 입력 방향에 따라 공격하는 방향이 달라집니다. 상하좌우 네 방향으로 공격 가능하며, 대신 지면에 서 있는 상태에서는 하단 공격이 불가능합니다. 또한 하단 공격으로 특정 장애물 혹은 적을 공격하면 튕겨나듯이 공중으로 잠시 솟아오르는 기능이 있습니다. 공격 방식이 하단공격인 경우에만 작동합니다. velocity.y 값을 12f로 설정하는 방식으로 구현되었습니다. 공격 애니메이션은 방향에 따라 캐릭터 기준으로 회전시켜 방향을 변경하며, trigger를 애니메이션에 맞춰 켜고 끄는 형태로 작동합니다.	
개발 중 발생한 문제 1.캐릭터의 이동 방향이 반대가 되면 상단 공격과 하단 공격이 반대로 출력되는 문제	
해결방안 1.캐릭터가 바라보는 방향에 따라 변화하는 localScale.x (1 or -1)를 곱해주어 상, 하단 공격 방향이 제대로 출력되도록 수정	

기술 요약	미니맵
모작의 대상인 게임에는 실시간으로 표시되는 미니맵이 없어 이를 구현해 보았습니다. 플레이어 오브젝트에 자식으로 와이드 카메라를 하나 넣어둔 뒤, RawImage를 통해 미니맵을 구현했습니다. 오브젝트에 색상 sprite를 포함시켜 맵 및 상호작용 오브젝트를 생성하고 이를 이용하여 미니맵 내에 출력해줍니다. 미니맵의 경우 디버그 모드를 이용해 두 가지 방식으로 토글이 가능하도록 제작하였습니다. 토글 버튼을 누를 때 마다 카메라의 컬링 마스크를 Minimap, Minimap을 제외한 전부 혹은 Nothing으로 변경하는 방식으로 표시되는 이미지를 변경시킵니다.	
	
간소화  색상으로 간소화된 미니맵을 표시합니다. 지면: 회색, 플레이어: 녹색, 상호작용: 핑크, 장애물: 적색, 포탈: 청색	
게임 자체 이미지  실제 플레이 화면을 그대로 축소시킨 화면을 표시합니다. 또한 토글을 통해 미니맵을 끄는 것도 가능합니다.	

기술 요약	벽 미끄러짐
	
플레이어는 미끄러질 수 있는 Slidable 태그 오브젝트의 벽을 짚을 수 있습니다. 플레이어는 벽을 짚고있는 동안 중력의 영향을 절반만 받습니다. 벽을 짚은 후 점프하는 경우 평소와 같이 방향키에 따라 점프하는 것이 아닌 강제로 짚은 벽의 반대 사선방향으로 점프하게 됩니다. 사선으로 점프할 시 특정 거리까지 점프하기 전에는 이동키를 눌러도 작동하지 않으며 해당 거리까지 도달한 이후부터 방향키가 작동하게 됩니다. 2단 점프는 거리에 상관없이 작동합니다. 해당 동작 방식은 원작을 그대로 구현한 것입니다.	
개발 중 발생한 문제 1.미끄러질 수 있는 벽에서 아래로 미끄러지지 않고 벽에 붙어있는 문제	
해결방안 1.Physics Material이 None인 경우 friction의 디폴트 값이 3D는 0.6, 2D는 0.4로 설정되어 있다는 것을 확인하고 미끄러질 수 있는 벽의 경우 friction을 0으로 수정한 Physics Material 2D를 추가	

기술 요약	자체 중력 시스템을 이용한 이동 및 점프 / 회피
	gravityScale에 의존하지 않고 velocity만을 이용하여 중력을 구현했습니다. 이렇게 이용했을때의 장점은 중력값을 디테일하게 수정하기 용이하기 때문입니다. 지면을 향한 ray를 쏘아 지면을 판별하며, 지면에서 붙지 않은 경우 $-9.81 * 4 * dt$를 velocity.y에 매 프레임마다 더하여 gravityScale에 의존하지 않는 중력을 구현하였습니다. 다양한 입력 방식을 구현해보기 위해 키 입력 제어에 input system을 이용했습니다. 플레이어는 2단점프와 대쉬를 기본적으로 가지고 있습니다. 점프의 경우 키를 누른 시간에 따라 점프하는 높이가 정해지며 키를 떼는 시점까지 점프하고 추락합니다. 점프는 최대 높이가 존재하며 해당 높이에 도달시 그 지점에서 즉시 추락합니다. 2단 점프의 경우 공중에서 점프를 한번 더 누른 경우 작동합니다. 이 경우도 일반 점프와 같이 점프 키를 떼 시점까지 점프하게 됩니다. 또한 지면에 닿기 전까지 변수가 초기화되지 않아 공중에서는 단 한번만 추가로 점프가 가능하도록 작성했습니다. 단, 슬라이딩 점프의 경우 2단 점프로 인정되지 않도록 하여 슬라이딩 점프는 무한으로 가능하며, 중간에 2단 점프를 수행하면 그 이후부터는 다시 지면에 닿을때까지 2단 점프가 불가능합니다. 대신 여전히 슬라이딩 점프는 가능합니다. 회피(대시)의 경우 rigidbody의 freezePosition y 값을 활성화하고 끄는 방식으로 작동됩니다. 키를 누르면 빠르게 바라보는 방향으로 직진합니다. dashSpeed 만큼 dashTime동안 이동하는 것으로 구현하였습니다. 대시가 끝난 후 고정된 y값을 해제하여 추락 혹은 추가 점프가 가능하도록 하였습니다. 대시중에는 이동 혹은 점프가 불가능 하도록 제한하여 대시가 끝날 때 까지 다른 동작을 수행하지 못하도록 하였습니다. 부적을 수정하고 플레이어의 리스폰 위치를 저장하는 벤치 오브젝트가 있습니다. 벤치에 다가가면 위 키를 입력할 시 앉을 수 있습니다. 앉은 상태에서는 velocity.y 값을 0으로 조정하여 추락하지 않고 그 위치에앉아있을 수 있도록 하였으며, 앉은 상태에서 아무 키를 입력하면 앉은 상태에서 벗어날 수 있습니다. 앉은 상태에서는 i키를 입력하면 나오는 부적 창에서 부적을 수정가능하도록 하였습니다. 모든 동작은 모작의 기반이 된 본편의 기획에 맞춰 제작하였습니다.
개발 중 발생한 문제	1.점프 시 천장에 닿은 경우 지정된 시간만큼 공중에서 머무르는 문제 2.공중에서 대쉬한 이후 마저 솟아오르거나 너무 빠른 속도로 추락하는 문제
해결방안	1.점프시 천장이 감지되면 즉시 velocity를 0으로 설정 2.대쉬가 완료된 시점의 velocity.y 값을 0으로 설정하여 추락하는 속도를 초기화

기술 요약	코드 통제 (디버그모드) / 시스템 최적화
	
디버그 모드를 통해 게임의 모든 부분에 있어 코드 통제가 되고 있으며 동적으로 활용할 수 있다는 것을 확인할 수 있습니다. D 버튼을 눌러 켜거나 끌 수 있습니다.	
보스 사망상태 제어 보스는 한번 사망시 playerpref 로 영구 사망 처리를 하게 됩니다. 따라서 다시 보스전을 치르고 싶은 경우 보스 부활 버튼을 통해 보스를 부활시킬 수 있습니다. 단, 보스 스테이지에 이미 진입한 경우 맵을 벗어났다 다시 진입해야합니다.	
플레이어 체력 관리 플레이어의 체력을 실시간으로 관리할 수 있습니다. 체력을 깎거나 회복할 수 있습니다. 또한 플레이어 사망 버튼을 통해 즉시 모든 체력을 깎고 부활 지점으로 이동할 수 있습니다.	
맵 이동 보스룸으로, 시작지역으로 버튼을 통해 맵 이동이 가능합니다. 맵을 이동하면 플레이어가 해당 맵으로 가는 포탈을 이용했을 때 스폰되는 지점으로 워프합니다.	
재화 획득 부족한 재화를 쉽게 수급하기 위한 재화를 제공하는 버튼을 이용할 수 있습니다. 재화를 획득하는 함수를 그대로 이용하기 때문에 재화 숫자가 올라가는 애니메이션 또한 동일하게 작동합니다. 버튼 1 회당 10 의 재화를 획득하며 연타하여 획득해도 문제가 발생하지 않습니다.	
미니맵 토글 미니맵을 켜다 켜거나 미니맵 타입을 변경할 수 있습니다. 간소화 버전, 실제 화면 버전, 고기를 순환합니다.	

재화 혹은 체력의 경우 매 프레임 갱신하는 경우 불필요한 값 갱신 및 대입을 반복하게 될 것이라고 판단하여, 이전 값을 저장해두고 변화가 생긴 경우에만 값을 적용하도록 하였습니다. 멀리 본다면 메모리 최적화 등의 성능 향상을 기대할 수 있을 것입니다.

또한 체력의 경우 이미지를 전체적으로 다시 그리지 않고 필요한 부분만 변경해줌으로써 UI 내에서의 변화를 최소화 하는 방식으로 최적화를 꾀했습니다.